

Arrays in Java: Insert Operations

This example builds a simple array class called **MyArray** with 5 slots. All slots start at 0. It supports inserting at the start, at the end, and at a given position, and printing every element with its index.

Instance variables

- **array**: stores the elements
- **length**: total size of the array (5)
- **currentIndex**: the next free position. It also equals the number of elements stored. When it equals length, the array is full.

Edge cases handled

- Inserting into a full array
- Inserting at a negative position
- Inserting at a position beyond currentIndex (this would leave a gap)

1. MyArray.java

```
public class MyArray {

    int[] array;        // stores the elements
    int length;         // total size of the array
    int currentIndex;    // next free position (also = number of elements)

    public MyArray() {
        length = 5;
        array = new int[length]; // Java fills it with 0 by default
        currentIndex = 0;
    }

    // Insert at the end
    public void insertAtEnd(int value) {
        if (currentIndex == length) {
            System.out.println("Array is full. Cannot insert " + value);
            return;
        }
        array[currentIndex] = value;
        currentIndex++;
    }

    // Insert at the start
    public void insertAtStart(int value) {
        if (currentIndex == length) {
            System.out.println("Array is full. Cannot insert " + value);
            return;
        }
        // shift all elements one step to the right
        for (int i = currentIndex; i > 0; i--) {
            array[i] = array[i - 1];
        }
        array[0] = value;
        currentIndex++;
    }

    // Insert at a given position
    public void insertAtPosition(int position, int value) {
        if (currentIndex == length) {
            System.out.println("Array is full. Cannot insert " + value);
            return;
        }
        if (position < 0 || position > currentIndex) {
            System.out.println("Invalid position " + position);
        }
    }
}
```

```

        + ". Valid positions are 0 to " + currentIndex);
    return;
}
// shift elements from position onwards one step to the right
for (int i = currentIndex; i > position; i--) {
    array[i] = array[i - 1];
}
array[position] = value;
currentIndex++;
}

// Print index and value of every element
public void printElements() {
    System.out.println("Index\tValue");
    for (int i = 0; i < length; i++) {
        System.out.println(i + "\t" + array[i]);
    }
    System.out.println("Current index: " + currentIndex);
    System.out.println();
}
}

```

2. ArrayDemo.java

```

public class ArrayDemo {

    public static void main(String[] args) {
        MyArray myArray = new MyArray();

        System.out.println("Initial array:");
        myArray.printElements();

        myArray.insertAtEnd(10);
        myArray.insertAtEnd(20);
        System.out.println("After inserting 10 and 20 at the end:");
        myArray.printElements();

        myArray.insertAtStart(5);
        System.out.println("After inserting 5 at the start:");
        myArray.printElements();

        myArray.insertAtPosition(2, 15);
        System.out.println("After inserting 15 at position 2:");
        myArray.printElements();

        // invalid positions
        myArray.insertAtPosition(-1, 99);
        myArray.insertAtPosition(7, 99);

        myArray.insertAtPosition(4, 25);
        System.out.println("After inserting 25 at position 4 (array is now full:)");
        myArray.printElements();

        // array is full, all of these should fail
        myArray.insertAtEnd(30);
        myArray.insertAtStart(1);
        myArray.insertAtPosition(2, 50);
    }
}

```

3. How to run

```
javac MyArray.java ArrayDemo.java
java ArrayDemo
```

4. Output

Initial array:

Index	Value
0	0
1	0
2	0
3	0
4	0

Current index: 0

After inserting 10 and 20 at the end:

Index	Value
0	10
1	20
2	0
3	0
4	0

Current index: 2

After inserting 5 at the start:

Index	Value
0	5
1	10
2	20
3	0
4	0

Current index: 3

After inserting 15 at position 2:

Index	Value
0	5
1	10
2	15
3	20
4	0

Current index: 4

Invalid position -1. Valid positions are 0 to 4

Invalid position 7. Valid positions are 0 to 4

After inserting 25 at position 4 (array is now full):

Index	Value
0	5
1	10
2	15
3	20
4	25

Current index: 5

Array is full. Cannot insert 30

Array is full. Cannot insert 1

Array is full. Cannot insert 50